

Ride-Sharing System Design with PostgreSQL

Table of Contents

1. [Overview](#)
 2. [Requirements](#)
 3. [Capacity Estimation](#)
 4. [Schema Design](#)
 5. [ASCII ER Diagram](#)
 6. [Indexing Strategy](#)
 7. [Query Patterns](#)
 8. [Scaling Strategy](#)
 9. [Tradeoffs](#)
 10. [Interview Discussion Points](#)
 11. [Common Interview Follow-ups](#)
 12. [Performance Considerations](#)
 13. [Cross-References](#)
-

Overview

A ride-sharing system is one of the most complex real-time database designs because it must reconcile three different time scales: millisecond location updates, second-level trip state transitions, and minute-level pricing adjustments. This document focuses on the system architecture — specifically how PostgreSQL (with PostGIS) handles the persistent data while Redis handles the real-time state. The schema expands on

`18_Architecture_Case_Studies/03_uber_ridesharing.md` with a focus on the matching engine, trip state machine, and real-time updates architecture.

Requirements

Functional Requirements

- Real-time driver location tracking (every 4 seconds)
- Trip request: rider selects vehicle class, sees estimated fare
- Matching engine: find and notify nearest available driver
- Trip state machine: request → assigned → en-route → arrived → in-progress → completed
- Dynamic surge pricing
- In-app navigation for drivers
- Real-time trip tracking for riders
- Rating and tipping after trip
- Driver earnings and weekly payouts

Non-Functional Requirements

- 5M drivers, 100M riders globally
 - 200K location updates per second at peak (800K active drivers × 1 update/4s)
 - Sub-500ms driver matching (from request to driver notification)
 - 15M trips per day
 - Sub-100ms location query response
 - 99.99% uptime for trip requests
 - Location data: operational retention 30 days; anonymized 2 years
-

Capacity Estimation

Metric	Value
Daily trips	15M
Peak trip requests per second	500
Driver location updates per second	200K
Active drivers (global peak)	800K
Cities served	500+
Surge pricing updates per minute	5K (10 zones × 500 cities)
Ratings submitted per day	30M

Storage by Component

Component	Technology	Data Volume
Driver locations (live)	Redis	800K × 200B = 160MB (tiny)
Trip records	PostgreSQL	~5GB/day, ~1.8TB/yr
Trip events	PostgreSQL	~35GB/day, ~12TB/yr
Payment records	PostgreSQL	~3GB/day, ~1TB/yr
Location history (trip paths)	PostgreSQL	~2GB/day, ~730GB/yr
Analytics events	Kafka + Warehouse	~10GB/day

Schema Design

```
-- =====
-- EXTENSIONS
-- =====
CREATE EXTENSION IF NOT EXISTS "uuid-osspl";
CREATE EXTENSION IF NOT EXISTS "postgis";
CREATE EXTENSION IF NOT EXISTS "postgis_topology";

-- =====
-- REAL-TIME STATE (Redis, mirrored here for persistence)
-- =====
-- Redis key: geo:available:{city_id} → GEOADD {lng} {lat} {driver_id}
-- Redis key: driver:{driver_id}:state → HASH {status, vehicle_id, city_id, ...}
-- Redis key: trip:{trip_id}:state → HASH {status, driver_id, rider_id, ...}
-- Redis key: surge:{zone_id}:{vehicle_class} → FLOAT multiplier

-- PostgreSQL stores canonical/durable state
-- Redis stores live state for fast access
```

```

-- =====
-- GEOGRAPHIC ZONES
-- =====

CREATE TABLE cities (
    city_id          SERIAL          PRIMARY KEY,
    name             VARCHAR(200)    NOT NULL,
    country_code     CHAR(2)         NOT NULL,
    timezone         VARCHAR(60)     NOT NULL,
    boundary         GEOGRAPHY(MULTIPOLYGON, 4326),
    is_active        BOOLEAN         NOT NULL DEFAULT TRUE,
    default_currency CHAR(3)         NOT NULL DEFAULT 'USD'
);

CREATE TABLE zones (
    zone_id          SERIAL          PRIMARY KEY,
    city_id          INT             NOT NULL REFERENCES cities(city_id),
    name             VARCHAR(200)    NOT NULL,
    zone_type        VARCHAR(50)     NOT NULL DEFAULT 'standard',
    boundary         GEOGRAPHY(POLYGON, 4326) NOT NULL
);

-- =====
-- DRIVERS
-- =====

CREATE TYPE driver_status AS ENUM ('offline', 'available', 'on_trip', 'in_break');
CREATE TYPE vehicle_class AS ENUM ('economy', 'comfort', 'premium', 'xl', 'pet',
'accessibility');

CREATE TABLE drivers (
    driver_id        UUID            PRIMARY KEY DEFAULT uuid_generate_v4(),
    email            VARCHAR(320)    NOT NULL,
    phone            VARCHAR(30)     NOT NULL,
    full_name        VARCHAR(200)    NOT NULL,
    status           driver_status   NOT NULL DEFAULT 'offline',
    avg_rating       NUMERIC(3,2)    NOT NULL DEFAULT 5.0,
    rating_count     INT             NOT NULL DEFAULT 0,
    total_trips      INT             NOT NULL DEFAULT 0,
    city_id          INT             REFERENCES cities(city_id),
    created_at       TIMESTAMPTZ     NOT NULL DEFAULT now(),
    CONSTRAINT uq_driver_email UNIQUE (email)
);

CREATE TABLE vehicles (
    vehicle_id       UUID            PRIMARY KEY DEFAULT uuid_generate_v4(),
    driver_id        UUID            NOT NULL REFERENCES drivers(driver_id),
    class            vehicle_class   NOT NULL,
    make             VARCHAR(100),
    model            VARCHAR(100),
    year             SMALLINT,
    plate_number     VARCHAR(20)     NOT NULL,
    capacity         SMALLINT        NOT NULL DEFAULT 4,

```

```

        is_active          BOOLEAN          NOT NULL DEFAULT TRUE,
        CONSTRAINT uq_plate UNIQUE (plate_number)
    );

-- UNLOGGED for ephemeral live location (5x faster writes, acceptable data loss on
-- crash)
CREATE UNLOGGED TABLE driver_locations (
    driver_id      UUID          PRIMARY KEY REFERENCES drivers(driver_id),
    location        GEOGRAPHY(PPOINT, 4326) NOT NULL,
    heading         SMALLINT,
    speed_kmh       SMALLINT,
    city_id         INT          REFERENCES cities(city_id),
    updated_at      TIMESTAMPTZ  NOT NULL DEFAULT now()
);

-- =====
-- PRICING
-- =====
CREATE TABLE pricing_rules (
    rule_id         SERIAL        PRIMARY KEY,
    city_id         INT          NOT NULL REFERENCES cities(city_id),
    vehicle_class   vehicle_class NOT NULL,
    base_fare       NUMERIC(8,2) NOT NULL,
    cost_per_km     NUMERIC(8,4) NOT NULL,
    cost_per_minute NUMERIC(8,4) NOT NULL,
    min_fare        NUMERIC(8,2) NOT NULL,
    booking_fee     NUMERIC(8,2) NOT NULL DEFAULT 0,
    effective_date  DATE         NOT NULL,
    CONSTRAINT uq_pricing UNIQUE (city_id, vehicle_class, effective_date)
);

CREATE TABLE surge_pricing (
    zone_id         INT          NOT NULL REFERENCES zones(zone_id),
    vehicle_class   vehicle_class NOT NULL,
    multiplier      NUMERIC(4,2) NOT NULL DEFAULT 1.0,
    demand_index    NUMERIC(6,2),
    supply_index    NUMERIC(6,2),
    updated_at      TIMESTAMPTZ  NOT NULL DEFAULT now(),
    PRIMARY KEY (zone_id, vehicle_class),
    CONSTRAINT chk_surge CHECK (multiplier BETWEEN 1.0 AND 8.0)
);

-- =====
-- TRIPS (Core Table)
-- =====
CREATE TYPE trip_status AS ENUM (
    'requested', 'searching', 'driver_assigned', 'driver_en_route',
    'arrived', 'in_progress', 'completed', 'cancelled', 'no_driver'
);

CREATE TABLE trips (
    trip_id         UUID          NOT NULL DEFAULT uuid_generate_v4(),

```

```

rider_id            UUID            NOT NULL,
driver_id           UUID            REFERENCES drivers(driver_id),
vehicle_id          UUID            REFERENCES vehicles(vehicle_id),
vehicle_class       vehicle_class  NOT NULL,
city_id             INT             REFERENCES cities(city_id),
status              trip_status    NOT NULL DEFAULT 'requested',

-- Locations
pickup_location     GEOGRAPHY(POINT, 4326) NOT NULL,
pickup_address       VARCHAR(500),
pickup_zone_id      INT            REFERENCES zones(zone_id),
destination_location GEOGRAPHY(POINT, 4326),
destination_address  VARCHAR(500),
destination_zone_id  INT            REFERENCES zones(zone_id),
actual_route         GEOGRAPHY(LINESTRING, 4326),

-- Measurements
distance_km          NUMERIC(8,3),
duration_minutes     NUMERIC(8,2),

-- Timing
requested_at         TIMESTAMPTZ    NOT NULL DEFAULT now(),
driver_assigned_at   TIMESTAMPTZ,
driver_arrived_at    TIMESTAMPTZ,
trip_started_at      TIMESTAMPTZ,
trip_ended_at        TIMESTAMPTZ,
cancelled_at         TIMESTAMPTZ,

-- Fare
estimated_fare       NUMERIC(10,2),
surge_multiplier     NUMERIC(4,2)   NOT NULL DEFAULT 1.0,
final_fare           NUMERIC(10,2),
currency             CHAR(3)        NOT NULL DEFAULT 'USD',

-- Cancellation
cancel_reason        TEXT,
cancel_actor         VARCHAR(20),

PRIMARY KEY (trip_id, requested_at)
) PARTITION BY RANGE (requested_at);

CREATE TABLE trips_2024_q1 PARTITION OF trips
FOR VALUES FROM ('2024-01-01') TO ('2024-04-01');
CREATE TABLE trips_2024_q2 PARTITION OF trips
FOR VALUES FROM ('2024-04-01') TO ('2024-07-01');

-- Trip state machine audit log
CREATE TABLE trip_events (
  event_id           UUID            NOT NULL DEFAULT uuid_generate_v4(),
  trip_id            UUID            NOT NULL,
  status             trip_status    NOT NULL,
  actor              VARCHAR(20),

```

```

        location          GEOGRAPHY(POINT, 4326),
        metadata          JSONB,
        created_at        TIMESTAMPTZ      NOT NULL DEFAULT now(),
        PRIMARY KEY (event_id, created_at)
) PARTITION BY RANGE (created_at);

-- =====
-- MATCHING ATTEMPTS (for analytics and driver fairness)
-- =====

CREATE TABLE match_attempts (
    attempt_id            UUID              PRIMARY KEY DEFAULT uuid_generate_v4(),
    trip_id               UUID              NOT NULL,
    driver_id             UUID              NOT NULL,
    distance_m            INT,
    notified_at           TIMESTAMPTZ      NOT NULL DEFAULT now(),
    response              VARCHAR(20),     -- 'accepted', 'rejected', 'timeout'
    responded_at          TIMESTAMPTZ
);

-- =====
-- PAYMENTS AND EARNINGS
-- =====

CREATE TABLE payments (
    payment_id            UUID              PRIMARY KEY DEFAULT uuid_generate_v4(),
    trip_id               UUID              NOT NULL,
    rider_id              UUID              NOT NULL,
    amount                NUMERIC(10,2)    NOT NULL,
    currency              CHAR(3)          NOT NULL DEFAULT 'USD',
    status                VARCHAR(20)      NOT NULL DEFAULT 'pending',
    provider              VARCHAR(50)      NOT NULL,
    provider_txn_id       VARCHAR(200),
    created_at            TIMESTAMPTZ      NOT NULL DEFAULT now(),
    CONSTRAINT uq_payment_trip UNIQUE (trip_id)
);

CREATE TABLE driver_earnings (
    earning_id            UUID              PRIMARY KEY DEFAULT uuid_generate_v4(),
    driver_id             UUID              NOT NULL REFERENCES drivers(driver_id),
    trip_id               UUID              NOT NULL,
    gross_fare            NUMERIC(10,2)    NOT NULL,
    platform_fee          NUMERIC(10,2)    NOT NULL,
    net_earning           NUMERIC(10,2)    NOT NULL,
    tip_amount            NUMERIC(10,2)    NOT NULL DEFAULT 0,
    currency              CHAR(3)          NOT NULL DEFAULT 'USD',
    created_at            TIMESTAMPTZ      NOT NULL DEFAULT now(),
    CONSTRAINT uq_earning_trip UNIQUE (trip_id)
);

-- =====
-- RATINGS
-- =====

CREATE TABLE ratings (

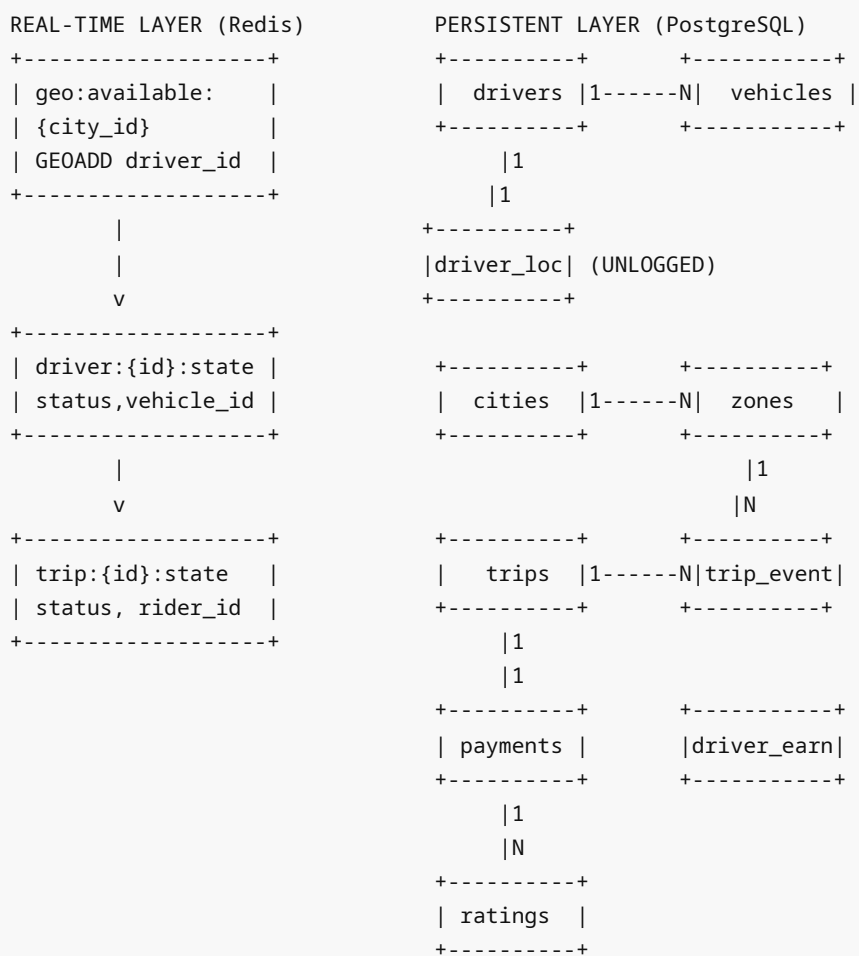
```

```

rating_id      UUID          PRIMARY KEY DEFAULT uuid_generate_v4(),
trip_id        UUID          NOT NULL,
from_rider     BOOLEAN       NOT NULL,
score          SMALLINT      NOT NULL,
comment        TEXT,
tags           TEXT[],
created_at     TIMESTAMPTZ    NOT NULL DEFAULT now(),
CONSTRAINT uq_rating UNIQUE (trip_id, from_rider),
CONSTRAINT chk_score CHECK (score BETWEEN 1 AND 5)
);

```

ASCII ER Diagram



Indexing Strategy

```

-- GEOSPATIAL (most critical for matching)
CREATE INDEX idx_driver_loc_gist ON driver_locations USING GIST (location);
CREATE INDEX idx_zones_boundary ON zones USING GIST (boundary);
CREATE INDEX idx_trips_pickup ON trips USING GIST (pickup_location);

```

```

-- Driver availability (pre-filter before geospatial)
CREATE INDEX idx_drivers_avail      ON drivers (status, city_id)
    WHERE status = 'available';

-- Trips: recent active trips
CREATE INDEX idx_trips_active      ON trips (status, requested_at DESC)
    WHERE status NOT IN ('completed', 'cancelled', 'no_driver');

-- Trips: rider history
CREATE INDEX idx_trips_rider      ON trips (rider_id, requested_at DESC);

-- Trips: driver history
CREATE INDEX idx_trips_driver      ON trips (driver_id, requested_at DESC)
    WHERE driver_id IS NOT NULL;

-- Surge pricing: zone lookup (hot read, fits in shared_buffers)
CREATE INDEX idx_surge_zone      ON surge_pricing (zone_id, vehicle_class);

-- Payments: status tracking
CREATE INDEX idx_payments_pending  ON payments (status, created_at)
    WHERE status = 'pending';

-- Earnings: driver payout computation
CREATE INDEX idx_earnings_driver  ON driver_earnings (driver_id, created_at DESC);

```

Query Patterns

Q1 — Find Nearest Available Drivers (PostGIS)

```

SELECT
    d.driver_id, v.vehicle_id, v.class,
    d.avg_rating,
    ST_Distance(
        dl.location,
        ST_SetSRID(ST_MakePoint($lng, $lat), 4326)::geography
    ) AS distance_meters,
    dl.heading, dl.updated_at
FROM driver_locations dl
JOIN drivers d ON d.driver_id = dl.driver_id AND d.status = 'available'
JOIN vehicles v ON v.driver_id = d.driver_id AND v.class = $vehicle_class AND
v.is_active
WHERE ST_DWithin(
    dl.location,
    ST_SetSRID(ST_MakePoint($lng, $lat), 4326)::geography,
    $search_radius_meters
)
AND dl.updated_at > now() - INTERVAL '30 seconds'

```

```
ORDER BY distance_meters ASC
LIMIT 20;
```

Q2 — Compute Fare Estimate

```
WITH pricing AS (
    SELECT base_fare, cost_per_km, cost_per_minute, min_fare, booking_fee
    FROM pricing_rules
    WHERE city_id = $city_id AND vehicle_class = $class
        AND effective_date <= CURRENT_DATE
    ORDER BY effective_date DESC LIMIT 1
),
surge AS (
    SELECT COALESCE(multiplier, 1.0) AS mult
    FROM surge_pricing
    WHERE zone_id = $zone_id AND vehicle_class = $class
)
SELECT
    p.base_fare,
    p.cost_per_km * $dist AS dist_component,
    p.cost_per_minute * $mins AS time_component,
    s.mult AS surge_mult,
    GREATEST(
        (p.base_fare + p.cost_per_km * $dist + p.cost_per_minute * $mins)
        * s.mult + p.booking_fee,
        p.min_fare
    ) AS estimated_fare
FROM pricing p, surge s;
```

Q3 — Assign Driver (Optimistic Concurrency)

```
BEGIN;

-- Only assign if trip is still in 'requested' state
UPDATE trips
SET status          = 'driver_assigned',
    driver_id       = $driver_id,
    vehicle_id      = $vehicle_id,
    driver_assigned_at = now()
WHERE trip_id = $trip_id AND requested_at = $requested_at
    AND status = 'requested';

-- Fail if 0 rows updated (another server assigned it first)
-- Application checks rowcount

UPDATE drivers
SET status = 'on_trip'
WHERE driver_id = $driver_id AND status = 'available';
```

```
INSERT INTO trip_events (trip_id, status, actor)
VALUES ($trip_id, 'driver_assigned', 'system');

COMMIT;
```

Q4 — Complete Trip

```
BEGIN;

UPDATE trips
SET status          = 'completed',
    trip_ended_at   = now(),
    distance_km      = $distance,
    duration_minutes = $duration,
    final_fare       = $final_fare,
    actual_route     = ST_GeomFromText($route_wkt, 4326)
WHERE trip_id = $trip_id AND requested_at = $requested_at;

UPDATE drivers
SET status = 'available', total_trips = total_trips + 1
WHERE driver_id = $driver_id;

INSERT INTO driver_earnings
    (driver_id, trip_id, gross_fare, platform_fee, net_earning)
VALUES
    ($driver_id, $trip_id, $final_fare, $platform_fee, $net_earning);

INSERT INTO payments
    (trip_id, rider_id, amount, status, provider)
VALUES
    ($trip_id, $rider_id, $final_fare, 'pending', $provider);

INSERT INTO trip_events (trip_id, status, actor)
VALUES ($trip_id, 'completed', 'system');

COMMIT;
```

Q5 — Rider Trip History

```
SELECT
    t.trip_id, t.status, t.requested_at,
    t.pickup_address, t.destination_address,
    t.final_fare, t.currency, t.distance_km, t.duration_minutes,
    d.full_name AS driver_name,
    v.make, v.model, v.plate_number,
    r.score AS rating_given
FROM trips t
LEFT JOIN drivers d ON d.driver_id = t.driver_id
LEFT JOIN vehicles v ON v.vehicle_id = t.vehicle_id
```

```

LEFT JOIN ratings r ON r.trip_id = t.trip_id AND r.from_rider = TRUE
WHERE t.rider_id = $1
      AND t.requested_at >= now() - INTERVAL '1 year'
ORDER BY t.requested_at DESC
LIMIT 20 OFFSET $2;

```

Q6 — Surge Pricing Computation (runs every 60 seconds)

```

-- Example: compute demand/supply for a single zone
WITH demand AS (
    SELECT
        pickup_zone_id AS zone_id,
        vehicle_class,
        COUNT(*) AS trip_requests
    FROM trips
    WHERE requested_at >= now() - INTERVAL '10 minutes'
          AND status IN ('requested', 'searching', 'driver_assigned')
    GROUP BY 1, 2
),
supply AS (
    SELECT
        z.zone_id,
        v.class AS vehicle_class,
        COUNT(*) AS available_drivers
    FROM driver_locations dl
    JOIN drivers d ON d.driver_id = dl.driver_id AND d.status = 'available'
    JOIN vehicles v ON v.driver_id = d.driver_id AND v.is_active
    JOIN zones z ON ST_Within(dl.location::geometry, z.boundary::geometry)
    GROUP BY 1, 2
)
INSERT INTO surge_pricing (zone_id, vehicle_class, multiplier, demand_index,
supply_index, updated_at)
SELECT
    z.zone_id,
    d.vehicle_class,
    LEAST(8.0, GREATEST(1.0,
        ROUND((1 + (d.trip_requests::FLOAT / NULLIF(s.available_drivers, 0) - 1) *
0.4)::NUMERIC, 1)
    )) AS multiplier,
    d.trip_requests,
    COALESCE(s.available_drivers, 0),
    now()
FROM demand d
JOIN zones z ON z.zone_id = d.zone_id
LEFT JOIN supply s ON s.zone_id = d.zone_id AND s.vehicle_class = d.vehicle_class
ON CONFLICT (zone_id, vehicle_class) DO UPDATE
    SET multiplier    = EXCLUDED.multiplier,
        demand_index = EXCLUDED.demand_index,
        supply_index  = EXCLUDED.supply_index,
        updated_at    = EXCLUDED.updated_at;

```

Q7 — Driver Earnings Summary

```
SELECT
    DATE_TRUNC('week', de.created_at) AS week,
    COUNT(*) AS completed_trips,
    SUM(de.gross_fare) AS gross,
    SUM(de.net_earning) AS net,
    SUM(de.tip_amount) AS tips,
    de.currency
FROM driver_earnings de
WHERE de.driver_id = $1
    AND de.created_at >= now() - INTERVAL '8 weeks'
GROUP BY 1, 6
ORDER BY 1 DESC;
```

Q8 — City Analytics: Trips by Zone

```
SELECT
    z.name AS zone_name,
    COUNT(*) AS trip_count,
    AVG(t.final_fare) AS avg_fare,
    AVG(t.distance_km) AS avg_distance
FROM trips t
JOIN zones z ON z.zone_id = t.pickup_zone_id
WHERE t.city_id = $1
    AND t.requested_at >= $from AND t.requested_at < $to
    AND t.status = 'completed'
GROUP BY z.zone_id, z.name
ORDER BY trip_count DESC;
```

Q9 — Update Driver Location

```
INSERT INTO driver_locations (driver_id, location, heading, speed_kmh, city_id,
updated_at)
VALUES (
    $driver_id,
    ST_SetSRID(ST_MakePoint($lng, $lat), 4326)::geography,
    $heading, $speed, $city_id, now()
)
ON CONFLICT (driver_id) DO UPDATE
    SET location = EXCLUDED.location,
        heading = EXCLUDED.heading,
        speed_kmh = EXCLUDED.speed_kmh,
        updated_at = EXCLUDED.updated_at;
```

Q10 — Match Attempt Recording

```

INSERT INTO match_attempts (trip_id, driver_id, distance_m, notified_at)
VALUES ($trip_id, $driver_id, $distance, now())
ON CONFLICT DO NOTHING;

-- Update response when driver responds:
UPDATE match_attempts
SET response = $response, responded_at = now()
WHERE trip_id = $trip_id AND driver_id = $driver_id;

```

Scaling Strategy

Architecture: Layered Real-Time + Persistent

```

CLIENT (Driver App)
|
| Location update (every 4s)
v
LOCATION SERVICE (stateless, horizontal)
|
|-- Redis GEOADD (primary store for matching)
|-- Kafka (async logging to PostgreSQL driver_locations)
|-- PostgreSQL UNLOGGED table (secondary, for geofencing queries)

MATCHING SERVICE
|
| Redis GEORADIUS query (< 1ms)
v
PostgreSQL for confirmation + state transitions

```

Phase 1: City-Scale (1 city, 50K drivers)

- Single PostgreSQL + PostGIS
- Redis for driver locations (GEORADIUS) and trip state
- All tables in one schema

Phase 2: Multi-City (100+ cities, 1M drivers)

- City-based sharding: each major city (or region) has its own PostgreSQL instance
- A routing service maps `city_id` → `DB_shard`
- Cross-city trips (rare) handled by a coordination service
- Redis: separate geo-set per city (`geo:available:{city_id}`)

Phase 3: Global (500+ cities, 5M drivers)

- Region-based clusters (US, EU, APAC, LatAm)
- Each region has 3–5 PostgreSQL shards (by `city_id` hash)
- Kafka for cross-region analytics events
- Driver location: entirely Redis Geo (not PostgreSQL), replicated to regional Redis clusters
- Trips: written to regional PostgreSQL; globally replicated for analytics (Kafka → BigQuery)

Real-Time Tracking Architecture

Driver Location → Redis GEORADIUS for matching
→ Redis PubSub 'trip:{id}:location' for rider tracking
→ Kafka → PostgreSQL batch insert (trip_route_points)

Rider App:

- Subscribed to Redis PubSub channel for their trip
- Receives driver location every 4 seconds without polling PostgreSQL

Tradeoffs

Decision	Chosen	Alternative	Why
Redis for live locations	Redis GEORADIUS	PostGIS on PostgreSQL	Redis GEORADIUS: sub-millisecond, in-memory; PostGIS required for complex geofencing
UNLOGGED for driver_locations	UNLOGGED table	WAL-logged	5x faster writes; location is ephemeral — safe to lose on crash
Optimistic concurrency for assignment	WHERE status='requested'	SELECT FOR UPDATE	No lock held during network round-trip to notify driver; simpler failure path
Quarterly trip partitions	Quarterly	Monthly	Trips generate less volume than messages; quarterly is sufficient
Store route as LINESTRING	Single geometry	Point table	~20 sampled points per trip sufficient for display; point table would be 20× larger

Interview Discussion Points

- 1. Why is the driver location table UNLOGGED?** Driver locations are ephemeral — they expire within 30 seconds (stale location check). Losing them on a crash (PostgreSQL restarts clean) is acceptable because all drivers will re-send their location within 30 seconds. UNLOGGED tables skip WAL writes, giving ~5x write throughput improvement.
- 2. How does the matching algorithm work?** Step 1: Redis GEORADIUS query to find the 20 nearest available drivers within 5km. This takes sub-millisecond. Step 2: Filter by vehicle class, rating, acceptance rate. Step 3: Notify the top driver via push notification. If they don't respond within 15 seconds, notify the next. PostgreSQL records the match attempt and the final assignment.
- 3. How do you prevent two riders from being matched to the same driver?** The PostgreSQL `UPDATE drivers SET status='on_trip' WHERE driver_id=$1 AND status='available'` is atomic. The first transaction to execute this UPDATE wins. It also removes the driver from the Redis `geo:available:{city}` set. The second transaction gets 0 rows updated and moves to the next candidate.
- 4. How does surge pricing work technically?** Every 60 seconds, the pricing service runs Q6 to compute demand/supply ratio per zone. The result is UPSERTed into `surge_pricing`. This table is tiny (zones

× vehicle classes) and fits in PostgreSQL shared_buffers. Additionally, the pricing service caches the entire table in Redis with a 60-second TTL for sub-millisecond fare estimation.

Common Interview Follow-ups

Q: How would you implement driver batching (pool rides)? A: Add a `pool_group_id` UUID to trips. A pool coordinator service finds trips with pickup locations within 500 meters and destinations in the same direction (ST_Azimuth comparison). Matched trips share a `pool_group_id`. The driver is assigned to the pool group, not individual trips. Each rider's fare is reduced by 30–40%.

Q: How does real-time tracking work for the rider? A: WebSocket server subscribes to Redis PubSub channel `trip:{trip_id}:driver_location`. The location service pushes to this channel on every driver GPS update. The WebSocket server fans out to the connected rider client. No polling of PostgreSQL.

Q: How do you handle a driver going offline mid-trip? A: If a driver's location update stops arriving (no heartbeat for > 60 seconds during an in-progress trip), the trip is flagged as `status = 'interrupted'`. The rider is notified. A human review team resolves the trip (charge based on last known location or distance estimate).

Performance Considerations

- **GiST index on driver_locations geography:** Enables `ST_DWithin` to use a bounding box index scan ($O(\log N)$). Without it, every matching query does a full sequential scan of 800K rows.
 - **UNLOGGED + fillfactor=50:** `driver_locations` is 100% UPSERTs (updates in-place). `fillfactor=50` ensures HOT updates work without index maintenance on every driver heartbeat.
 - **Partition pruning:** Always include `requested_at >= $date` in trip queries. The partition key is `requested_at`; without this bound, all quarterly partitions are scanned.
 - **Surge pricing fits in cache:** `surge_pricing` has at most 10K rows (200 zones × 5 vehicle classes × 10 cities). It fits entirely in PostgreSQL shared_buffers and Redis. Every fare estimation is a cache hit.
 - **Connection routing:** Matching service makes thousands of geospatial queries/second. Direct to primary (writes). Reporting/analytics direct to read replica. PgBouncer pool sizes: matching pool = 100, analytics = 20.
-

Cross-References

- See `18_Architecture_Case_Studies/03_uber_ridesharing.md` for full schema details
- See `06_chat_application_design.md` for WebSocket + LISTEN/NOTIFY patterns
- See `08_system_design_framework.md` for interview presentation framework
- PostGIS: Chapter 11 of this guide
- UNLOGGED tables: Chapter 5 of this guide
- Partitioning: Chapter 5 of this guide